

ENBC321/BIOE658I: Homework #4

Due: Monday, 03/02/26 by 5:00 PM, beginning of the class

Submit your homework to ELMS, including Python code and the dataset

Overall points: 100

Note: Late homework may be submitted up to 24 hours after the due date, but a 50% penalty will be applied.

Note: Please aim to write clean, well-organized code that is easy for others to understand. Be sure to include comments or explanations where needed. Points may be deducted if this is not done.

Note: Please name your codes using the following format: `problem1_fullname.ipynb`. Start with the problem number and followed by your full name, e.g., **`problem1_jimmyazarnoosh`**.

Note: Using generative AI for learning purposes is allowed. However, directly copying or closely replicating AI-generated content is strictly prohibited and will result in a grade of ZERO. Repeated violations may lead to more severe consequences.

Note: Submit **a single zip file** containing the following:

- The dataset you used
- Python code files in ***.ipynb**
- Your answers to the questions in PDF format

Decision Trees vs. Random Forests

Problem 1. Implement **Decision Tree** and **Random Forest Regressors** in Python using **scikit-learn** to analyze their performance on a medical **Breast Cancer** dataset and answer conceptual questions about these models. The goal is to predict a continuous target variable related to breast cancer. (20 points)

Tasks: (10 points)

- Use the Breast Cancer Wisconsin dataset from **scikit-learn**:
`sklearn.datasets.load_breast_cancer()`
- Modify the dataset by selecting a continuous variable (mean radius) as the target variable.
- Split the dataset into 80% training and 20% testing.
- Train a Decision Tree Regressor (`DecisionTreeRegressor`) and a Random Forest Regressor (`RandomForestRegressor` with 100 trees).
- Evaluate both models using **Mean Squared Error (MSE)** and **R² score** on the test set.
- Train the Decision Tree with different values for `max_depth` (3, 5, 10) and `min_samples_split` (2, 5, 10). Analyze how these hyperparameters affect performance. Train the Random Forest with different values for `n_estimators` (10, 50, 200). Compare results.

Answer these questions:

1. Why might a Decision Tree Regressor overfit the training data? (2 points)
2. How does Random Forest help reduce overfitting? (2 points)
3. Compare the bias and variance of Decision Trees and Random Forests. (3 points)
4. What happens when `n_estimators` (number of trees in a Random Forest) is increased? (3 points)

Logistic Regression

Problem 2. In this problem, you implement **logistic regression** to predict whether a patient has diabetes based on health measurements. You will evaluate the model's performance, experiment with different decision thresholds, and answer conceptual questions. (20 points)

Tasks: (10 points)

- Use the diabetes dataset from **scikit-learn**.
`sklearn.datasets.load_diabetes()`
- Convert the continuous target variable into a binary classification (above/below median).
- Split the dataset into 80% training and 20% testing.
- Train a Logistic Regression model. Use `LogisticRegression()` to fit the model. Print the learned coefficients and intercept.
- Compute and display: Accuracy, Precision, Recall, and F1-score
- Default threshold for logistic regression is **0.5**. However, changing the threshold can impact model predictions. Try different thresholds (**0.3**, **0.5**, **0.7**) and observe how the model performance changes.

Answer these questions:

1. What happens to precision and recall when lowering the threshold from 0.5 to 0.3? (2 points)
2. When might a lower threshold be useful in a medical diagnosis setting? (2 points)
3. How would the model change if we used fewer features? (3 points)
4. Would you expect logistic regression to work well if we had 1000 features? Why or why not? (3 points)

K-Nearest Neighbors (KNN)

Problem 3. In this problem, you will implement the **K-Nearest Neighbors (KNN)** algorithm in Python using the diabetes dataset from **Scikit-learn**. A hospital is trying to classify patients as either having diabetes or not based on key medical features. (20 points)

Tasks: (10 points)

- Use the diabetes dataset from **scikit-learn**.
`sklearn.datasets.load_diabetes()`
- Convert the continuous target variable into a binary classification (above/below median).
- Split the dataset into 80% training and 20% testing.
- Train a KNN classifier with different values of K (e.g., 3, 5, 7).
- Compute and display: Accuracy, Precision, Recall, and F1-score

Answer these questions:

1. What happens if K is too small? What if K is too large? How does it affect bias and variance? (3 points)
2. The default metric for KNN is Euclidean distance. Would using Manhattan distance change the results? Why or why not? (3 points)
3. Our dataset was split into two classes based on the median. How would the results be affected if one class was much larger than the other? (4 points)